

Politecnico di Torino

Politecnico di Torino

Corso di Laurea Magistrale

A.a. 2026/2027

Large Language Models for Software Engineering

Docenti:
Flavio Giobergia
Riccardo Coppola

Studente:
Cristiano Berardo

Indice

1 Introduction to LLM 1
- 1.1 What is a Language Model (LM)? 1
  - 1.1.1 Formal definition for language models 1
- 1.2 Simple LMs - n-gram models 1
- 1.3 Estimation & generation 1
- 1.4 Generating new sentences 2
- 1.5 Limitation of n-grams 2
- 1.6 Bigger and better Language Models 2

2 Introduction to deep learning 3
- 2.1 The perceptron 3
- 2.2 The perceptron, in 2D 3
- 2.3 Activation functions 3
  - 2.3.1 Enforce properties on perceptron's output 4
  - 2.3.2 Adding some perceptrons and adding other layers! 4
  - 2.3.3 Introduce non-linearities in the model 4
- 2.4 Multi-layer perceptron models 4
- 2.5 Activation functions for classification models 4
- 2.6 Binary classification 5
- 2.7 Multi-class classification 5
- 2.8 Activation functions for regression models 5
- 2.9 Defining weights (parameters) 5
- 2.10 Linear regression 6
- 2.11 More complex losses/model 6
  - 2.11.1 Limitation of gradient descent 6
- 2.12 Backpropagation 6
  - 2.12.1 Using the chain rule 7
  - 2.12.2 Computational graph 7
- 2.13 Loss functions 8

3 Vector Embedding 9
- 3.1 Before word embeddings: One-Hot encoding 9
  - 3.1.1 Problems of One-Hot Encoding 9
- 3.2 Distributed representation 9
  - 3.2.1 Solving the right task 10
- 3.3 In terms of matrices 10
- 3.4 Limitation of WordToVec 10
- 3.5 FastText 10
- 3.6 Visualizations - Semantic meanings 10
- 3.7 Inside an RNN 10
  - 3.7.1 Weight sharing 11
- 3.8 Encoder-decoder architecture 11

4 Recurrent Neural Networks 12
- 4.1 Recurrent Neural Networks - RNN 12

5 Transformers 13
5.1 Simplified architecture . . . 13
5.2 Generation the first token . . . 13
5.3 Tokenization . . . 14
5.4 Byte-Pair Encoding - desiderata . . . 15
5.4.1 BPE results . . . 15
5.5 Some extra tokens . . . 16
5.6 Positional encoding . . . 16
5.7 Sinusoidal Positional encoding . . . 16
5.7.1 Positional encoding uniqueness . . . 17
5.8 Positional encoding preserves similarity . . . 17
5.8.1 Learned positional embeddings . . . 17
5.9 Attention . . . 17
5.9.1 Classic dictionary lookup . . . 17
5.9.2 Dot-product attention - Weighted lookup . . . 18
5.9.3 Attention as Keys/Values/Queries . . . 18
5.9.4 the 3-inputs block in the transformers' architecture . . . 18
5.9.5 Packing everything into matrices . . . 18
5.10 Scaled Dot-product attention . . . 19
5.10.1 Attention example . . . 19
5.10.2 Types of attention . . . 19
5.11 Multi-head attention . . . 19
5.12 Some additional details . . . 19
5.12.1 Residual connections . . . 19
5.12.2 Layer Normalization . . . 19
5.12.3 Actual connections . . . 20
5.12.4 Relative positional embeddings . . . 20
5.12.5 Advantages of transformers . . . 20
5.13 Encoder-decoder architecture . . . 20
5.13.1 Single model, multiple tasks! . . . 20
5.13.2 Not instruction-tuned, yet! . . . 21
5.14 Beyond encoder-decoder . . . 21
5.14.1 Encoder-only architecture . . . 21
5.14.2 BERT - Bidirectional Encoder Representations from Transformers . . . 21
5.14.3 Input encoding . . . 21
5.14.4 Masked LM . . . 22
5.14.5 Next Sentence Prediction . . . 22
5.14.6 Attention example in BERT . . . 22
5.14.7 Decoder-only architecture . . . 22
5.14.8 GPT (Generative Pre-trainined Transformer) . . . 22
5.15 Sampling approaches . . . 22
5.15.1 Some sampling approaches . . . 22

6 The GPT Family 24
6.1 GPT-1 (Generative Pre-trainined Transformer) . . . 24
6.1.1 GPT-1 Architecture & tasks . . . 24
6.1.2 GPT-1 setup . . . 24
6.2 GPT-2 . . . 25
6.2.1 GPT-2 setup . . . 25
6.2.2 Architecture . . . 25
6.3 Scaling Laws for Neural Language Models . . . 25
6.4 GPT-3 . . . 26
6.5 Fine -tuning vs in -context . . . 26
6.5.1 In-context performance . . . 26
6.5.2 Comparison with fine-tuned models . . . 26
6.5.3 The race to bigger models . . . 26
6.6 Big models got bigger . . . 27
6.6.1 Oversized and undertrained - explained . . . 27

6.7 Approach 1 (fixed-sized models) . . . 27
6.8 Chinchilla . . . 27
6.9 Llama family . . . 27

7 Metrics 28
7.1 Geometric mean (a digression) . . . 28
7.2 Perplexity . . . 28

Capitolo 1

Introduction to LLM

1.1 What is a Language Model (LM)?

A Language Model is a probabilistic model of a natural language. It produce a probability for any sequence of words.

The probability over all possible sentences in a language sums to 1. There are region of high density in the space of all possible sentences.

1.1.1 Formal definition for language models

$$P(w1, w2, ..., wn) = \prod_{t=1}^{T} P(w_t | w_1, w_2, ..., w_{t-1})$$

1.2 Simple LMs - n-gram models

We need to estimate the probability of a word given the previous words. A simplifying assumption, Markov assumption, is that:

$$P(w_t | w_1, w_2, ..., w_{t-1}) \approx P(w_t | w_{t-n+1}, ..., w_{t-1})$$

The next word w_t depends only on a small windows of $n-1$ preceding words called the **context**.

For $n = 2$, the next word only depends on the previous word, called a **bigram** model.

$$P(w_t | w_{t-1}, w_{t-2}, ..., w_1) \approx P(w_t | w_{t-1})$$

1.3 Estimation & generation

We can use a language model to estimate probabilities for new sentences. Generate a new sentence based on known probabilities.

- Starting from a given word, we can compute the probability of having any other word following it
- Then, we choose one word based on this probability distribution
- We add the new word to the already available sequence, and generate the next one
- We call this, autoregressive generation

Example $P(the, cat, chased, the, cheese) = P(the) \times P(cat|the) \times P(chased|the, cat) \times P(the|the, cat, chased) \times P(cheese|the, cat, chased, the)$

We can use the Markov assumption to simplify the computation of the probabilities.

$P(the, cat, chased, the, cheese) \approx P(the) \times P(cat|the) \times P(chased|cat) \times P(the|chased) \times P(cheese|the)$

Let's assume $P(the) = 1$

$P(the, cat, chased, the, cheese) \approx 1 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16}$

1.4 Generating new sentences

1.5 Limitation of n-grams

1. **Data sparsity**: As n increases, the number of possible n-grams grows exponentially. Bigram, V words, the table size is V^2. n-grams, V words, the table size is V^n. A small vocabulary or a small context size make the LM quite useless
2. **Context limitation**: Only local context (previous n-1 words) are used to choose. Longer sentences may require remembering what happened "early on"
3. **Lack of semantics**:Similar words are treated in the same way as completely different ones

The languafe models are not a new concept. In the '90s, researchers tried to build LMs based on existing techniques and computing capabilities. With varying results... Mostly bad.

1.6 Bigger and better Language Models

We can see $P(w_t|w_{t-1}, w_{t-2}, ..., w_1)$ as a classification function.

"Given an **input**, the previous words in the sentence, predict the **next word**, class."

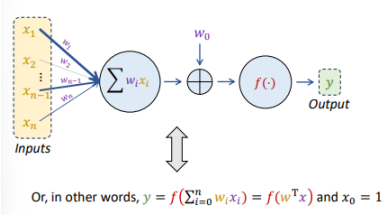
Capitolo 2

Introduction to deep learning

2.1 The perceptron

The perceptron is the simplest unit of **neural networks**. It takes an input with multiple features and **multiplies** it by a weight. Then, it sums all the weighted inputs and applies a non-linear function to produce an output.

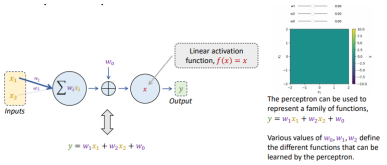
Where: $$y = f(w_0 + w_1x_1 + w_2x_2 + ... + w_nx_n)$$



- y is the output of the perceptron
- f represents a non-linear "activation" function
- w_i and w_0 are the weights and bias witch are learned during training
- $x = (x_1, x_2, ..., x_n)$ are the inputs to the perceptron

2.2 The perceptron, in 2D

If the f function is a linear activation function $f(x) = x$ then the equation is $y = w_1x_1 + w_2x_2 + w_0$.
Various values of w_0, w_1, w_2 defie the different functions that can be learned by the perceptron.
In this way perceptron can be used to represent a family of functions.

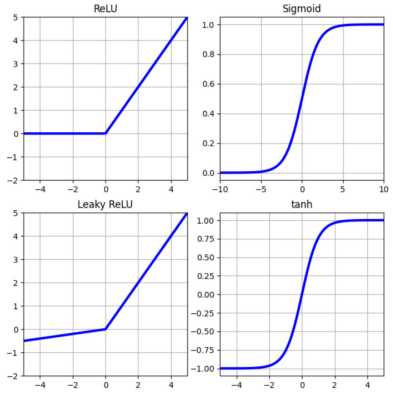


2.3 Activation functions

Activation functions are used for two main reasons:

1. Enforce properties on perceptron's output. E.g. sigmoid → binds output to [0, 1] range
2. Introduce non-linearities in the model + some others (faster convergence, sparsity, ...)

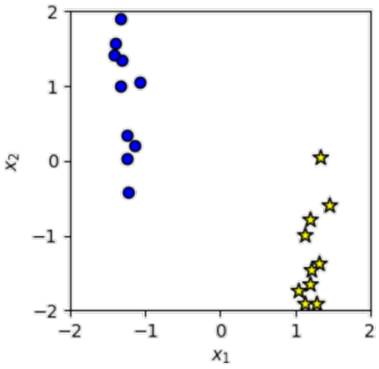
Common activation functions are: ReLU, Sigmoid, Leaky ReLU, Tanh, Softmax, Linear, GeLU



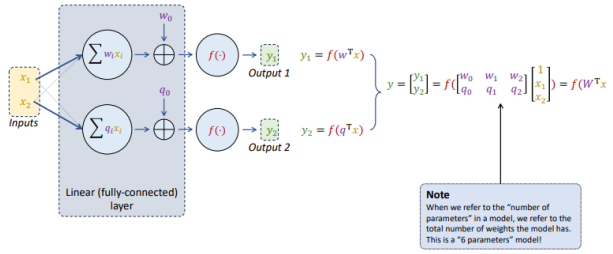
2.3.1 Enforce properties on perceptron's output

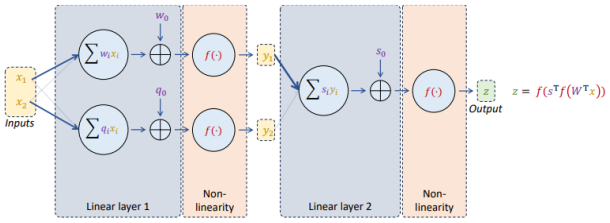
Binary classification problem: seprate positive ($\star$) and negative ($\circ$) samples. For a point $x \in \Re^2$, the perceptron can predict $p(\star|x)$. For a binary case this implies that $p(\circ|x) = 1 - p(\star|x)$.
To get a valid probability, we must enforce $p(\star|x) \in [0, 1]$. We already have $p(\circ|x) + p(\star|x) = 1$ by construction
The Sigmoid maps any value in $\Re$ to the range $[0, 1]$. The output is squashed between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



2.3.2 Adding some perceptrons and adding other layers!



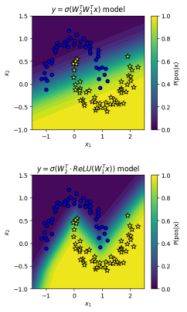


2.3.3 Introduce non-linearities in the model

This is the second reason why we need activation functions.
If $f(x) = x$ (i.e., no non-linearity is added) we get $z = s^T W^T x$
This implies:

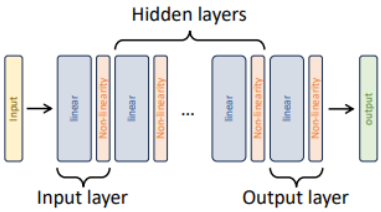
1. We could have used $W' = W_s$ and get the same output
2. We wouldn't have needed a second layer!
3. But our model is still linear

So, we use non-linear activation functions to model more complex functions



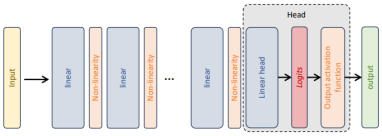
2.4 Multi-layer perceptron models

We can stack additional layers separated by non-linearities (activation functions) to prevent collapses.
Universal Approximation Theorem tells us that we can approximate "any" function with MLPs.
"For any continuous function g defined on a compact subset of $\Re^n$ and for any $\epsilon > 0$, there exists a feedforward neural network with a single hidden layer and a finite number of neurons that can approximate g to within an arbitrary degree of accuracy ϵ"
A single-layer MLP works ... but no information on the number of neurons, or the weights' values! Deeper, narrower networks are generally used

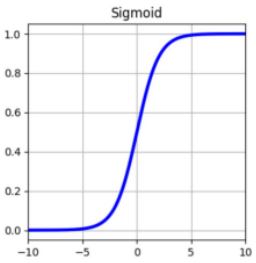


2.5 Activation functions for classification models

As argued, activation functions can be used to enforce properties on the model's output
In classification problems, the output before the final activation is treated as unnormalized probabilities **logits unit**.
We still need a step to convert logits into valid probabilities. E.g. all probabilities must sum to 1 and be in the range [0, 1].



2.6 Binary classification



The model predicts the probability of a single class for point x. As a convention, the positive one $P(pos|x)$.
The model produces a logit $z = model(x)$. We use the **Sigmoid function** on the output logit z: $\sigma(z) = \frac{1}{1+e^{-z}}$. This guarantees $P(pos|x)$ is in the range [0, 1].
We work out the probability of the negative class $P(neg|x) = 1 - P(pos|x)$.
We can easily show that $P(neg|x) \in [0, 1]$.
By construction, $P(pos|x) + P(neg|x) = 1$.

2.7 Multi-class classification



The output class is one of many classes (c_1, c_2, ..., c_n). The model produces n logit for a point x. I.e. the last layer have n perceptrons. $z = (z_1, z_2, ..., z_n) = model(x)$
We need to obtain, from the logits, valid probabilities. $P(c_1|x)$, $P(c_2|x)$, ..., $P(c_n|x)$.
The **softmax** function produce a probability of class (e.g. class 1 (cat), class 2 (dog)), it gives us only values between 0 and 1

$$P(c_i|x) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

2.8 Activation functions for regression models

In regression, models generally predict real numbers. Typically, there is no need to enforce properties. Output activation function can be the identity function $f(x) = x$. Generally the only situation where it makes sense to use it!

2.9 Defining weights (parameters)

So far, we assumed all weights and biases (let's call them θ) to be known. But, we still need to figure out how we find them!

We pick a function, onjective or loss, $\mathcal{L}(\theta)$, that we want to minimize.

in Linear Regression we minimize the Mean Squared Error

It looks at the model, the data are fixed, and give us a number, high number means bad model, low number means good model.

Example:

$$\mathcal{L}(\theta) = MSE(\theta) = \frac{1}{n}\sum(y_i - \theta^T x_i)^2$$

Then, we pick θ that minimizes it

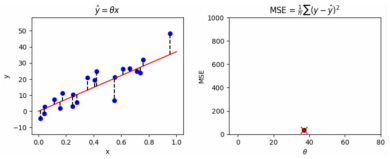
Note: $\mathcal{L}$ also depends on the training points x_i, y_i, so we should refer to it as $\mathcal{L}(\theta, X, y)$. However, the training set X, y is generally fixed. Thus, we only have control over θ, so we use the notation $\mathcal{L}(\theta)$.

2.10 Linear regression

For simple models, we can find the optimal weights in closed form

$$\frac{\partial \mathcal{L}(\theta)}{\partial \theta} = \frac{\partial MSE(\Theta)}{\partial \theta} = 0$$

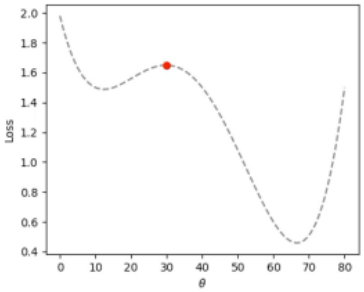
Quadratic in θ, can be solved easily!
Or, we can evaluate the loss function for a bunch of θ's, and find the "best" one



> **Note:** For linear regression, we don't try a bunch of θ's, since we can easily find the best value in closed form. However, this provides the intuition for what we will do next with more complex loss functions/models.

2.11 More complex losses/model

In more complex loss functions/models, we may not be able to solve the problem in a closed form. But we can evaluate $\mathcal{L}(\theta)$ for various values of θ.
We can iteratively update θ to reach a local minimum. We can start from a random value θ, then "move around" according to "some policy"
Move around = update θ incrementally, based on its current value. The new value of θ at any step depends on the previous step's value: $\theta_{t+1} := \theta_t + update$
Some policy = we take a small step in the direction where the function decreases locally. i.e. in the **opposite** direction of the **gradient**.
$\theta_{t+1} := \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t)$



For 1-dimensional θ, we have $\theta_{t+1} := \theta_t - \alpha \frac{\partial \mathcal{L}(\theta)}{\partial \theta}$
Where α: learning rate, controls the "size" of the step (10^{-3}, 10^{-4}, etc)
This is called **gradient descent** (GD). We can visualize it as "going down the hill" of the loss function.
If the derivative is negative, I move to the right, when the derivative is positive I move to the left and I get the bottom of the function.
Gradient Descent!

2.11.1 Limitation of gradient descent

GD is sensitive to weight initialization. Different initializations can lead to different solutions! GD can get stuck in local minima
Various solutions to help prevent local minima:

- Adding momentum
- Adaptive learning rates, how to regulate the step
- Learning rate schedules

2.12 Backpropagation

How to compute the gradient of a loss function with respect to the weights of a neural network?
So far, we assumed we were able to compute $\nabla_\theta \mathcal{L}(\theta)$.
We need some automated way to compute gradients.
Now we introduce the **backpropagation**, to compute the gradient of the loss w.r.t. any weight! Backpropagation is just a fancy word for "using the chain rule"

2.12.1 Using the chain rule

We use the chain rule from calculus

Other notation caould be $f(g(x))' = f'(g(x)) \cdot g'(x)$ $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g}\frac{\partial g}{\partial x}$

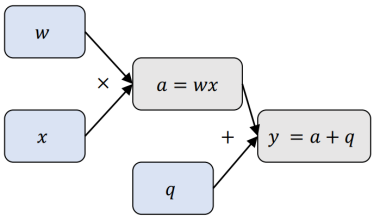
And apply it from the end of the computational graph, backwards. Hence the name, backpropagation.

2.12.2 Computational graph

A computational graph is a directed graph

- Each node corresponds to an operation
- Each edge represents the flow of data between nodes

For instance, we may want to compute $y = wx + b$

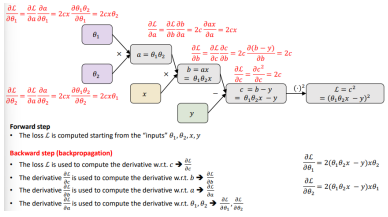


We start from three variables, w, x and y. The computational graph performs one operation at a time.

- First, compute the intermediate variable $a = wx$
- Then, compute the output variable $z = a + y = wx + y$

Backpropagation example Let's say:

- Out dataset has one point (x, y)
- Our (weird) model has two parameters, θ_1 and θ_2, and predicts $\theta_1\theta_2 x$
- Our loss function is $L = (\theta_1\theta_2 x - y)^2$



We build a computational graph with all operations and intermediate variables.

- $a = \theta_1\theta_2$
- $b = ax = \theta_1\theta_2 x$
- $c = b - y = ax - y = \theta_1\theta_2 x - y$
- $L = c^2 = (b - y)^2 = (ax - y)^2 = (\theta_1\theta_2 x - y)^2$

2.13 Loss functions

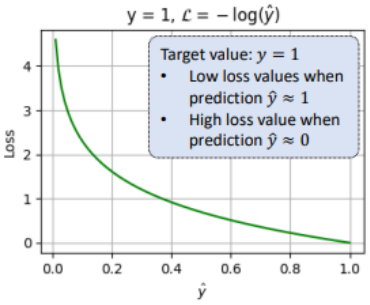
What loss functin should I use?

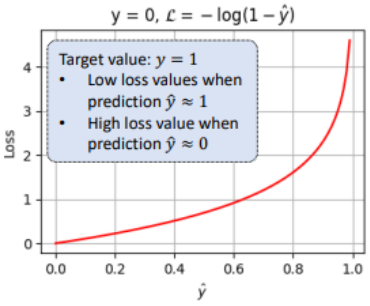
- **Regression**: Mean Squared Error (MSE), Mean Absolute Error (MAE)
- **Binary**: Binary Cross Entropy (BCE), the ground truth is $y = \{0, 1\}$ and the predicted value is $\hat{y} = model(x) \in [0, 1]$

$$\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

Where y is the ground truth, acts as a "selector" of the loss term to be applied.

$$y = 1 \rightarrow \mathcal{L} = -\log(\hat{y})$$
$$y = 0 \rightarrow \mathcal{L} = -\log(1 - \hat{y})$$





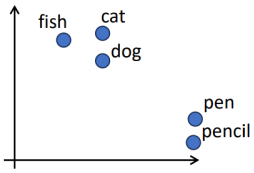
- **Multi-class classification**:
  - Cross-Entropy
  - Generalization to multiple classes of BCE
  - $yF = 1$ when ground truth is the ith class, 0 otherwise
  - yF plays the same "selector" mechanism as in BCE

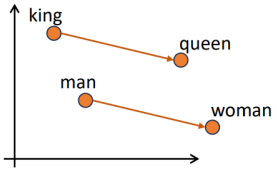
$$\mathcal{L} = -\sum_{i} y_i \log(\hat{y}_i)$$

Capitolo 3

Vector Embedding

Word embeddings are an advanced and **dense** vector representations of words, with a limited number of dimensions. Dense as opposed to sparse, e.g. one-hot encoding.
Each word is mapped to a **vector of real numbers**. High-dimensionalities, 300+ dimensions, are used to have enough space to represent various facets of words.
Word embeddings capture **semantic meanings** and **reletionships** between words.





- **Semantic meaning**: words with similar meanings have similar representations
- **Relationships**: words with similar connections are lined via similar transformations

3.1 Before word embeddings: One-Hot encoding

One-hot encoding was used before embeddings. A vocabulary of 5 words have 5 dimension vector where only one bit set to 1 and the rest are 0.
We assume a vocabulary W with $|W|$ words. For example, a vocabulary of 5 words: *cat, dog, fish, bird, mouse*, $|W| = 5$. An order can be established among words e.g. lexicographically.
One-hot encoding creates for each of the $|W|$ words, a $|W|$-dimensional sparse vector

```
dog    → [1 0 0 0 0]
cat    → [0 1 0 0 0]
fish   → [0 0 1 0 0]
pen    → [0 0 0 1 0]
pencil → [0 0 0 0 1]
```

3.1.1 Problems of One-Hot Encoding

- **Sparsity**: the vectors are sparse, this leads to scalability issues. A standard vocabulary can have 50K words, implying a 50K-dimensional vector for each word. The vector space is not used efficiently
- **Orthogonality**: the is no preservation of similaruty or relationships. Here, all pairs of words have exactly the same

3.2 Distributed representation

We distribute the sematic word across all the dimension. And one-hot encoding is a type of **local representation** where each entity is represented by a unique and isolated identifier.

By contrast, **distributed representation** aim to distruibute the information across several dimensions.

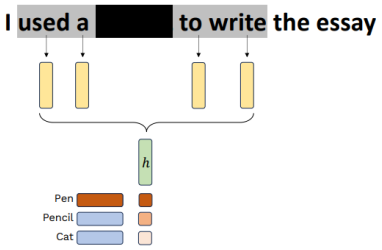
But how do we figure it the vector? We use NN model to learn there words and create this vectors.

We let models, e.g. neural networks, to learn these representations by craafcting a task and letting the model solve it.

E.g. "I use a ... to write the essay". We ask the model to predict the blank, this encourage the model to figure it out, starting the surrounded words called context. Can we figuire out the probability of each vocabulary, based on the context? Some step to

3.2.1 Solving the right task

1. assign a vector for each words in the context, at beginning is random number
2. Take all of this vector and aggregate all vectors and sum them up (**context vector**)
3. Assign a vector for each candidate output word, at beginning is random number
4. Compute the distance between the context vector and all possible output words (e.g., via dot product)
5. Find the word that best matches the context
6. Is it the correct word?
7. Adjust all vectors accordingly (via gradient descent)



The same process is applied to millions of sentences. Similar words are found in similar contexts and to solve the previous task, the word vectors of similar words must be simiar.

E.g. "I used a **pencil** to write the essay" and "You used my **pen** to write a letter"

Cosine similarity = cross entropy → gradient descent → backpropagation → update the vectors to make the loss function go dow.

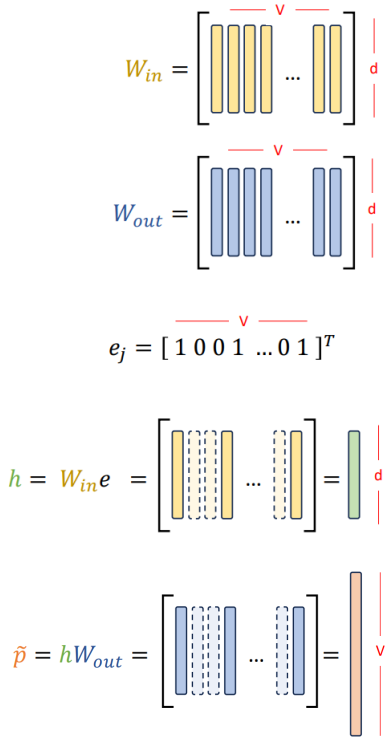
In this is reaped for millions of sentences. Pence or pencil now are similar and connected, semantic related, now two word can have same number, but it is fine.

3.3 In terms of matrices

We need to compute a matrix moltiplication. All vectors for input and output words can be staked into matrix $W_{in}(d \times V)$ and $W_{out}(d \times V)$. The entire context can be represented a binary vector of presences, $e(V \times 1)$.

> **Note:** This loses the order of the words, **bag of words** approach.

$h = W_{in}e(d \times 1)$ computes the sum of the vectors for the context words. Where e_j acts as a selector of the jth column within W_{in}.



3.4 Limitation of WordToVec

- Cannot manage words are not in the vocabulary, OOV (out of vocabulary)
- This vector are not contextualized

After the tranign, the vector is the same, and during the infference some word depends on the context, e.g. bank (river or money).

3.5 FastText

Allows to figure out some new words. Braking up words into subwords. E.g. <where> –> <wh, whe, her, ere, re> Insufficeint, the "in" has different meaning between two words and the prefixes of a word.

Does this works well? Not the perfect solution.

3.6 Visualizations - Semantic meanings

Reducing to 2 dimensions with Principal Component Analysis (for visualization purposes). The words belonging to the 3 categories are well-separated in the "compressed" embedding space

3.7 Inside an RNN

We have three dense NN layers. Two input and two output.

- **Input layer**: processes the current input, produces a vector with the same dimensionality as the hidden state. Makes the input and the state the same size dimentionality
- **State layer**: processes the hidden state from the previous step
- **Output layer**: Summed the two vectors and produces

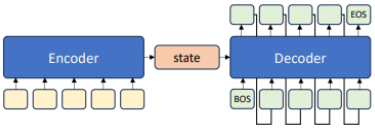
3.7.1 Weight sharing

Backpropagation-through-time

3.8 Encoder-decoder architecture

Ideally, the model should:

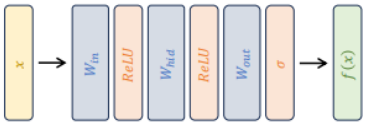
- See the entire sequence before starting generating the output



Capitolo 4

Recurrent Neural Networks

Limitations of Fully-Connected Neural Networks (FCNNs). They have some inputs and some output layers, interlieved by some hidden layers.



This type of models have some architectural constraints

- The input/feature is expected to be fixed in size, beacuse each neurons has a specific set of weights
- Each input/feature is associated with its set of weights
- The output/feature is expected to be fixed in size

The input is expected to be fixed in size

It is ok when is structured, such as a table. We only manage missing values. For unstructured and dynamic length data, such as images or text.

Each input is associated with its set of weights

If the same pattern occurs in different portions of the input, the model needs to learn the same pattern multiple times.

The output is expected to be fixed in size

As defined by the number of rows in W_{out}. Sometimes, we may want outputs to have varying sizes. E.g. In object detection, we want to segment the pixels of an image

4.1 Recurrent Neural Networks - RNN

Type of NN designed to press sequential data. It receave and looks at one element at a time. And to let the model to remember what it has seen before, we add a hidden state to summarizes the info seen so far.

RNN architecture

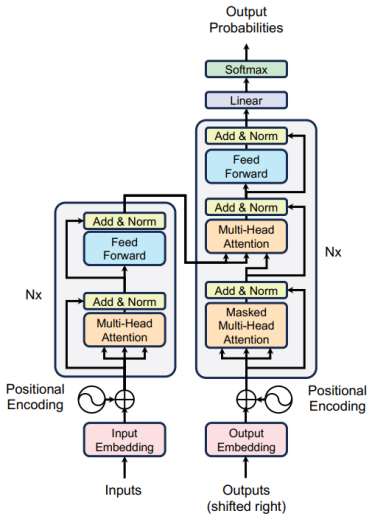
The general RNN architecture is charaterized by two **inputs** and two **outputs**:

- The first input is the current element of the sequence
- The second input is the hidden state from the previous time

RNN architecture, unfolded

Capitolo 5

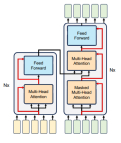
Transformers



Transformers architecture are not longer based on RNN. We shot the model the entire sequence wery puerfull, but is expencive.

5.1 Simplified architecture

We need yellow imputes and the model give us the green outputs.



We have two types of inputs:

- FF - Feed forward

5.2 Generation the first token

The input sequence pass to encoder model and the output of the encoder producers a vector for each token provided in input. 10 input 10 output, is a property of the attenction layer, this means no compression in a single vector and no loss of information if it is a good encoder.

The decoder takes the output of the encoder and we need to speciify the first token, we give the special vector. The model will look it and decide then it prdoduce output 1 input 1 output (BOS). The output is always a probabiilty function in our vocaboly. That word become the second word on the decoder input. The encoder knows the intire previous sequence, at the end we produce a end of sequence token (EOS).

Note: The decoder will output a prediction for each input token. At inference time, we are interested the latest generated token. At training time, the loss is computed on all tokens generated at that step!

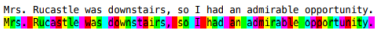
Cosality masking, 2 input 2 output, but the first token LLM should not see on the GUI, it already printed.

5.3 Tokenization

We need a way to split a sentence into units. This process is called tokenization, and splits the sequence into tokens

Transformers need to process natural language, in the form of sentences, braking into small units.

Character-level



Braking up the sentence into characters. Each character is a token.

Pros:

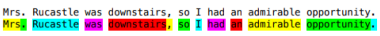
- Simple solutions
- No out-of-vocabulary problem
- Robust to misspellings and variations

Cons:

- Long sequences - transformers, attention mechanisms, are $O(N^2)$ in the sequence length
- Slower/more complex training
- Semantic information is harder to capture
- Inefficient for common words

Word-level

Breaks text into individual words. Each word is treated as as token



Pros:

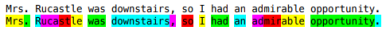
- Captures semantic meaning directly
- Shorter sequences compared to character-level
- More intuitive/easier to understand

Cons:

- Out-of-vocabulary (OOV) issues for rare or new words
- Does not leverage some information shared among words (e.g. prefixes/suffixes)
- Larger vocabulary needed, leading to memory inefficiency
- Vectors for rarer words are trained “less”

Subword-level

Breaks text into subword units (e.g., prefixes, suffixes). fastText uses n-grams contained in each word. A more commonly used method is Byte-Pair Encoding (BPE)



Pros:

- Balances between character-level and word-level
- Handles out-of-vocabulary words effectively
- Compact vocabulary with better generalization
- Efficient for both frequent and rare words

Cons:

- Common words are represented by fewer subwords w.r.t. rarer words
- Requires defining a subword definition policy
- The subword definition policy may introduce some computational overhead

In practice, subword-level is the most commonly adopted tokenization method. A simple, yet commonly adopted technique is BPE - Byte-Pair Encoding.

5.4 Byte-Pair Encoding - desiderata

Data driven approach is used to chose "best" tokens. To do so, we need a colection of text called **corpus**.

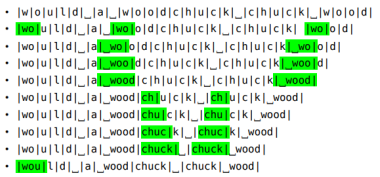
- Common words to be encoded into a single token
- Rare words to be encoded into multiple tokens

Also we difeine the desired maximum number of tokens

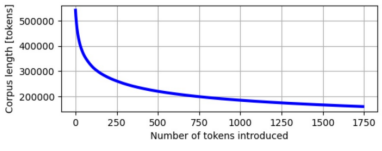
1. The corpus is initially encoded with 1 carachers byte = 1 token so we address all possible sentences.
2. We count the frequency of each pair of subsequent tokens is countd in the copus
3. The mosr frequent pair of tokens (F_1, F_2) is assiged a new token T
4. All occurrences of the pair (F_1, F_2) in the corpus are replaced with T
5. The process 2-4 is repeated until the desired number of tokens is reached

BPE example : The corpus we we would use as example is "would a woodchuck chuck wood"

- Corpus: "would a woodchuck chuck wood"
- Initial encoding: w | o | u | l | d | | a | | w | o | o | d | c | h | u | c | k | | c | h | u | c | k | | w | o | o | d
- Count frequency of pairs: (w, o): 3, (o, o): 2 ...
- Assign a new token to the most frequent pair: (w, o) → wo



5.4.1 BPE results



The number of tokens used to represent the corpus decreases as we increase the number of tokens used. There is a reasonable small number of tokens that can be used to represent a large corpus.

Typically, 10's to 100's of thousands of tokens are used

5.5 Some extra tokens

We introduce some special tokens to indicate that special positions within a sequence. For instance, we can use tokens for beginning/end of sequence during training.

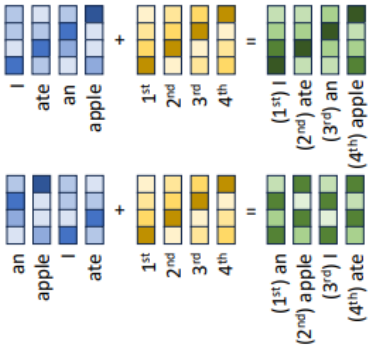
- BOS - Beginning of Sequence
- EOS - End of Sequence
- CLS - Classification token for task-specific use
- SEP - Separator token within the same input. E.g. Separate Question from Answer in Question Answering tasks

5.6 Positional encoding

Every time we have a token is associated to a vector learned during the training, start with random vectors, and adjust them via gradient descent.

The **same token** is mapped to the **same vector** regardless of its position in the sequence. This is not ideal. Additionlly the **Attentiona and Feed Forward does not understand the order** of the elements.

We have to figure it out a way to inject the position before passing it to the transformer. This is called **positional encoding**.



5.7 Sinusoidal Positional encoding

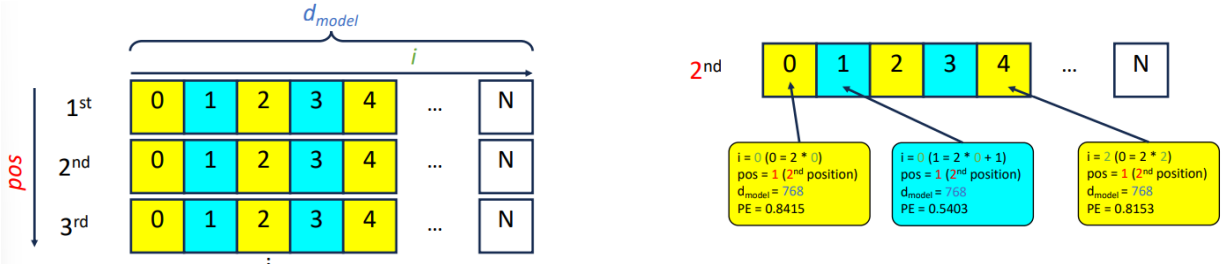
In the paper "Attention is all you need" Google engineers proposed a sinusoidal positional encoding.

In the AIAYN paper, the PE vectors are defined as:

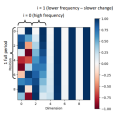
$$PE_{(pos,2i)} = sin(\frac{pos}{10000^{2i/d_{model}}})$$

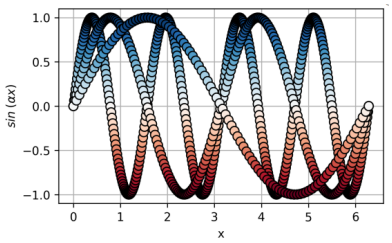
$$PE_{(pos,2i+1)} = cos(\frac{pos}{10000^{2i/d_{model}}})$$

where pos is the position of the token we have some dependencies. 10000 is a magic number often used in smaller transformer architecture. i the position of the dimension we are filling. d_{model} is the dimension of the model, how big are the vectors.



Error: in the blue box the formula should be $i = 1$





5.7.1 Positional encoding uniqueness

The vector for each position is unique... At least for the first $60K$ positions ($2\pi \cdot 10000$). then they start repeating (for longer sequences, we can just change the $10K$ constant!)

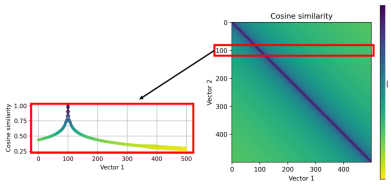
5.8 Positional encoding preserves similarity

If position 1 is close to position 2 (e.g., the third vs the fourth words in a sentence) we want positional vectors to also be close! We can easily verify that using trigonometric functions ensures that this is the case

Example for max position = 1,000 and dmodel = 768

Cosine similarity of each pair of the previous positional vectors

Maximum similarity along the diagonal (of course)



5.8.1 Learned positional embeddings

After the first transformer paper, other people proposed other solutions for positional encoding. Some model e.g. GPT family uses posisitona embeddings. This approach was also considered in AIAYN, but discarded as it did not produce better results

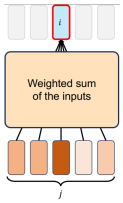
5.9 Attention

"How much" of each other vector is added is defined by the attention paid to them. Ideally, useful tokens will be weighted a lot. Unuseful tokens will be weighted less

The transformer learns to choose these weights based on the input sequence

Weight sum of the input = Attention

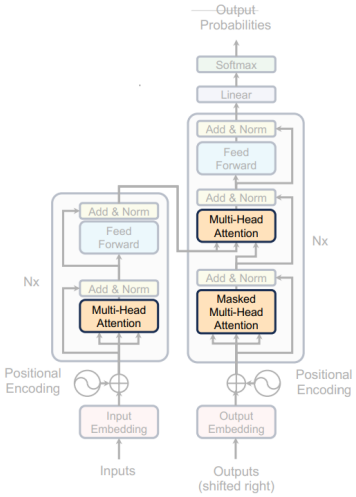
The attention module requires: An input (to be contextualized) and A way to define the attention weights The i-th output is defined as



$$\text{out}_i = \sum_j \text{Attn_weight}_{i,j} \cdot f(\text{in}_j)$$

5.9.1 Classic dictionary lookup

A small digression



5.9.2 Dot-product attention - Weighted lookup

5.9.3 Attention as Keys/Values/Queries

We expect to have multiple queries with three seprate "inputs":

- Queries: the value to be looked up, we want 1 output fro each query
- Keys: the values to be matched against the queries
- Values: the values associated to each key

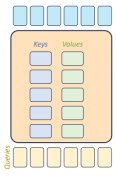
This is why the 3-inputs block in the transformers' architecture

5.9.4 the 3-inputs block in the transformers' architecture

How to to compute the attention weights? Since the dot function produce unbounded values, we need to normalize them. We can use the softmax.

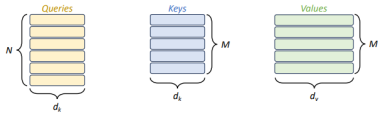
5.9.5 Packing everything into matrices

We discussed the situation having a single query. Of course, we can have multiple queries, each treated in the same way



For convenience, we can summarize the set of all queries, keys and values as three matrices, Q, K, V.

- Q: contains N queries, each being a d_k-dimensional vector
- K: contains M entries, each being a d_k-dimensional vector
- V: contains M entries (one value for each key), each being a d_v-dimensional vector output



5.10 Scaled Dot-product attention

In the original transformers, the final attention is defined as:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

- QK^T: efficiently computes the dot product between each query and each key. This produces an $N \times M$ matrix

5.10.1 Attention example

5.10.2 Types of attention

Attention is used in three situations in the classic transformer architecture.

The first one is called the encoder self-attention, Decoder self-attention, and the third one is called encoder-decoder cross attention.

Encoder self-attention

Decoder (masked) self-attention

This is the block used by the decoder. Used on the output tockens.

It looks only what has been generated so far, and not the future tokens.

Encoder-decoder cross-attention

Type of attention where part of the input comes from the encoder and part comes from the decoder.

The decoder produce the output and the cross attention, keys and values come from the encoder.

5.11 Multi-head attention

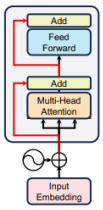
Each of the block before are multi-head attention, with its own W^Q, W^K, W^V . Since the attention may need to focus on different aspects in different contexts, we generally adopt multiple attention heads in parallel.
So far we discussed the attention mechanism with a single head.
The final attention output is the concatenation of the various heads' outputs. Passed through a linear layer to go back to the desired vector size.



5.12 Some additional details

5.12.1 Residual connections

Residual connections are a technique used to improve gradient flow in backpropagation.
The gradient "skips" part of the network when being backpropagated.
Indeed, these are sometimes called skip connections.
Is a shortcut to make it easyer non-zero signal to flow through the network.
The output is the sum of the input plus the input $out = f(x) + x$

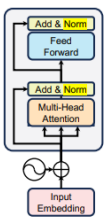


5.12.2 Layer Normalization

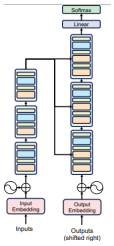
Add is the residual part, and Nord is the normalization layer. Such as single token vector, we normalize that across all deminsion a specific variance and a speicifi
This makes the training faster and more stable.

$$y = \frac{x - E[x]}{\sqrt{Var[x] + \epsilon}} \cdot \gamma + \beta$$

Where γ and β are learnable parameters, and ϵ is a small constant to avoid division by zero.

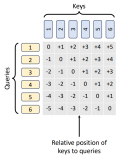


5.12.3 Actual connections



The encoder and decoder are stacked in this way. The output of the final encoder block becomes the input of the cross attentions of all the decoder.
The cross attention only takes the output of the last encoder block.
This means no match between encoder and decoder block, e.g. encoder can » decoder, or vice versa. Normally are the same number of blocks, but not necessary.

5.12.4 Relative positional embeddings



AIAYN uses sinusoidal absolute positional encoding, other, like GPT family, uses learned absolute positional embeddings.
This means there are different type of positional encoding.
Relative positional embeddings: one position of a sentence maps to one vector, but some people proposed to use relative positional changes for each words and the words near it.
Positional information no longer introduced in the input vectors, but at "attention time". We build vecrtor with a fixed size window.
Not applicable at the beginning but after compute the key score, when we have the relation.
Nowadays we use a mix of all the techniques.

5.12.5 Advantages of transformers

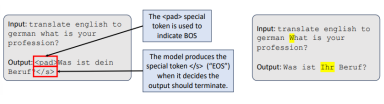
- **Parallelization**: Transformers can process all tokens simultaneously, no dep on previous states. Remember, we use teacher forcing to "know the future" - no need to wait for autoregressive generation
- **Long-range relationships**: With attention, Transformers can see the entire sequence at all times, and choose what to pay attention to. The model does not need to remember because it receave all the input context and use the attention mechanism to focus on the relevant parts. This implies that each token looks at all tokens, this is N2 and represents one of the current limitations of Transformers.
- **Better memory**: Since there is no sequential processing, forgetting issues generally do not occur

5.13 Encoder-decoder architecture



The famous example is the T5 - Text-to-Text Transfer Transformer.
This ia a vanilla endoder-decoder transformer. **Single framework**, same weights, for multiple tasks.
Summarization, translation.
How does it works?
We have one sentence and we add a prefix to the input, e.g. "summarize: " or "translate" to predend or append to the text, learned to the model.
Task prefixes to condition the decoder's output, instead of using different architectures.

5.13.1 Single model, multiple tasks!



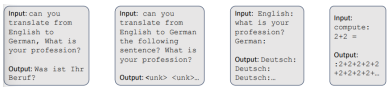
Note: All model have their representation of EOF, in this case we have <
s>

5.13.2 Not instruction-tuned, yet!

While it may resemble the behavior of instruction-tuned models ("chat-like conversation"), that's not it!. T5 is only tuned on specific tasks. The model learns to recognize those tasks and addresses them.

No strong generalization capabilities to new tasks

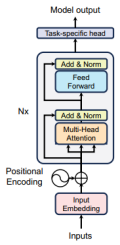
This means the model is not instruction-tuned! It is an initial model, but can solve multiple things.



5.14 Beyond encoder-decoder

5.14.1 Encoder-only architecture

Encoder-only architectures primarily focus on understanding and encoding input data "without generating output".
The "decoder" part of the architecture is removed. The encoder is trained to solve some tasks.
We have an input and want to do someting with this seuence e.g. classification, regression, name endity recognition, sentimental analysis, etc. and NOT want to generate a new sequence.



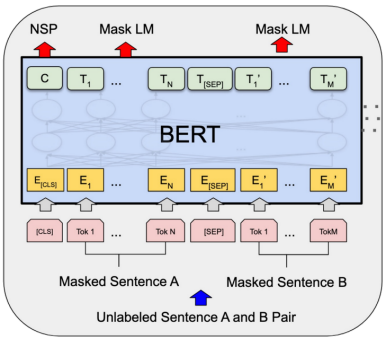
5.14.2 BERT - Bidirectional Encoder Representations from Transformers

This is arguably the most famous encoder-only transformer architecture.

It is pre-trained on two self-supervised tasks.

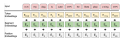
Pre-trained did the initial traingin on this task, but we can do the fine-tuning chanign a little bit more the weights depends on out task. This process needs only few hundred sentences.

It uses bidirectional attention the endoer self-attention.



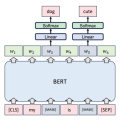
5.14.3 Input encoding

We have a token vector for each tocken, a positional embedding for each position and segment embedding with two vector A and B used for question and answer.
We intruduce the **segment embeddings**, they are two vectors, used by BERT allows to pass two sentences to the input of the model. This is usefull because traiinig is easyer and second one is allows the qeustion and answer



5.14.4 Masked LM

Google taken some sentences and removed some words, putting blanks. This is 15% of the words. The output of BERT is used to reconstruct the masked tockens.
This classification task is called Masked LM. We predict the masked tokens using the linear and softmax activation functions.



5.14.5 Next Sentence Prediction

This second task provides two sentences as input A and B. Is a self supervised task.
We ask BERT if the sentence B have sense after sentence A. We train the model to havve understanding the consequntiality.
Here one prediction for each sentence.
We add an additonal tocken at the beginning of the sentence called CLS, and we use the output vector as the source the information.
The limitation we cannot store of an entire paragraph in a vector, we need to compress, but this way is a good starting point.



5.14.6 Attention example in BERT

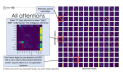
Esample to encode a simple sentence: "[CLS] The dog ate the food because it was hungry [SEP]".

BERT's attentions help understand how each token is considered throughout the transformer.

How many attention do we have? BERT use 12 tranformers layers one after the other. Each layer has 12 heads meaning 12 application of the attention mechanism. The sentence ha 11 tokens. Eanch head produces 11×11 attention map.

For a total of 12 layers times 12 heads = 144 attention maps. Each map is a 11×11 matrix.

The attention bean paied to next token, and some time paid attention previous token. The vector weight is influenced by the dog word.



5.14.7 Decoder-only architecture

The most interesting part used today.

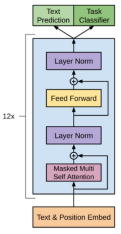
Is a type arch in witch we make a model extending a sequence. Just give the next token. The rest of the output is generated in an **autoregressive** manner.

We do not have the encoder, the decoder already sees the input as the beginning of the output.

We dont ave the cross-attention, we use only the decoder self-attention. We use cross-attention when we use multimodel model. GPT model family is a decoder-only architecture.

5.14.8 GPT (Generative Pre-trainined Transformer)

Decoder only transformer **pretrained** on the text generation task. Then, the model is **fine-tuned** on specific, supervised task. The model is provided the beginning of a sequence as the initial "decoder output".
Greedy sampling taks the most probable token at each step.



5.15 Sampling approaches

In order to chose the best token we need to use some teqniques. We have different stragegies usaully used together. In some cases, we want **deterministic** behaviors, in other cases we want **stochastic** behaviors.

5.15.1 Some sampling approaches

- **Greedy sampling**: Select the token with the highest probability. It is deterministic. Bead beahavior when the model read repatitive sequences (repetition penalty)

- **Beam search**: Still deterministic. Keep some option and explore the most promising ones. We use k how many parallel sentence adding the word
- **Random sampling**: Semantically has bit confuse
- **Top-k sampling**: I want to sample but not want long tails that has low prob. If high entropy very distributed.
- **Top-p sampling**: We want sample high prob. Instad of having a fixed k number of tokens, we select flexible k. The probability of the selected tokens is greater than p (95% prob mass).
- **Temperature sampling**: lika random sampling changing the softmax. If T is 0 means greedy sampling (deterministic) case.

Capitolo 6

The GPT Family

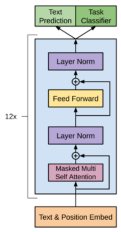
Evalution of GPT models:

- GPT-1 (2018): First in a (long series) of models. Decoder-only transformer architecture. Pretraining on a large corpus, fine-tuning on various tasks
- GPT-2 (2019): Larger training set. Initial controversies around the (lack of) release for potential misuse
- GPT-3 (2020): Scaling up: 175B parameters & more data!
- GPT-4 (2023): OpenAI no longer providing information on the model.

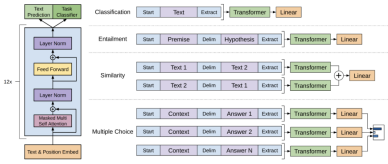
6.1 GPT-1 (Generative Pre-trainined Transformer)

Decoder-only transformer, pretrained on the text generation task. Training done with:

- **Unsupervised pretraining**: next tocken prediction, can be done on large datasets
- **Supervised fine-tuning**: on supervides datasets such as sentiment analysis. Typically done on smaller datasets, this is more expensive.



6.1.1 GPT-1 Architecture & tasks



6.1.2 GPT-1 setup

It was unsupervised pre-trained on the BooksCorpus dataset (800M words), 7K books of various genres. Approximately 5GB of text.

The fine-tuning was done using:

- Natural languafe inference
- Question answering
- Semantic similarity
- Text classification

The architecture of this model is:

- 12 layers decoder-only with 12 heads each
- 768 dimensdimensional state
- BPE with 40K merges
- Leared positional embeddings
- Context window of 512 tokens (WHAT!!?)
- 117M parameters

6.2 GPT-2

Realised in 2019, based on GPT-1 recipe, with lager training corpus (10x) and large model (10x). It was trained with unsupervised pretraining with no **fine-tuning**.

GPT-2 shows emergent capabilities. It could solve problems it was not explicitly trained on!

"Increasing [model capacity] improves performance in a log-linear fashion across tasks"

"Due to concerns about large language models being used to generate deceptive, biased, or abusive language at scale, we are only releasing a much smaller version of GPT-2 along with sampling code"

6.2.1 GPT-2 setup

Only unsupervised pretraining. Now they used WebText dataset (more data but less quality). Text from 45M links (40GB of text), semi-curated results - links from Reddit posts with at least 3 upvotes.

Fine-tuning is no longer used.

6.2.2 Architecture

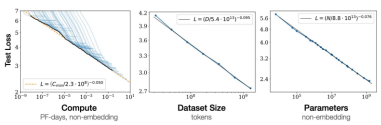
- Same as GTP-1
- BPE with 50257 tokens
- Context window of 1024 tokens
- Up to 1.5B parameters

6.3 Scaling Laws for Neural Language Models

Published in January 2020, by OpenAI. Shows various empirical takeaways. There are some scaling laws and the parameters are:

- Model size (number of parameters)
- Dataset size (number of tokens)
- Compute budget (number of FLOPs)

This is not a so obvious result, of cource it can be expected, but with that paper they show the formula that can be used to predict the performance of a model.



The Test Loss tell us how a model is performing on unseen data.

Other takeaways The model size is what is matter in that years. They see that fixing the number of parameters of a model and scaling the transformer depth, width are not as effective as scaling the model size.

The second thing is there is a law:

$$N \propto C^{0.73}, B \propto C^{0.24}, S \propto C^{0.03}$$

Where N = model size, B = batch size, S = number of steps, C = computing budget.

"As the computational budget C increases, it should be spent primarily on larger models, without dramatic increases in training time or dataset size".

10x more computing budget → 5.4x model size, 1.7x batch size, 1.07x training steps

6.4 GPT-3

Jennuray in 2020 published the paper we have already discussed, than in june 2020 OpenAI released GPT-3. It has 175B parameters, 10x more than GPT-2 with no other meaningful changes.
It was trained on CommonCrawl, after filtering, WebText2, Books1, Books2, Wikipedia
It bacame a generalistic model.



6.5 Fine -tuning vs in -context

How do we solve problems before GPT-3?

Fine-tuning Update model weights on a task-specific dataset. Previously considered the go-to approach. No fine -tuning done on GPT -3 in -context learning only!

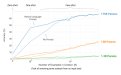
In-context learning this is new way to solve problems beacuse it has seen a lot of exampple and the solution is already into the weights. Model weights no longer updated. Task described as a part of the prompt in natural language.
Now instaed of translating a sentence from English to French using fine-tuning, we pass a desccription of the task.



- **One-shot**: No other information provided to the model. It understands, simply
- **One-shot**: One input/output example of provided in addition to task description
- **Few-shot**: Multiple examples of input/output pairs provided. Limited by maximum context size available

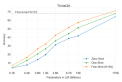
6.5.1 In-context performance

Larger models show remarkable 1- and few-shot performance w.r.t. smaller ones. The gap between 0and 1-shot performance is quite remarkable



6.5.2 Comparison with fine-tuned models

The 175B few-shot model is comparable (or outperforms) fine-tuned SOTA models. In the plot: TriviaQA, but similar results on other problems. This is a remarkable result: Scaling the model allows not fine-tuning on task-specific datasets and still get competitive results!



6.5.3 The race to bigger models

These scaling laws resulted in a race toward building larger models. GPT-3 was the first 100B+ parameters model. Other larger models have been developed, following this trend



6.6 Big models got bigger

After the OpenAI paper lots of company has started to chase the trend of building larger models. Reaching 500+B parameters in 2 years.

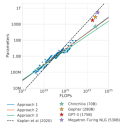
This until 2022 when DeepMind published "Training Compute-Optimal Large Language Models".

"current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping [...] data constant".

DeepMind claimed: "for every doubling of model size the number of training tokens should also be doubled"

6.6.1 Oversized and undertrained - explained

DeepMind runs additional experiments with 3 approaches.
All results indicate that the conclusions of Kaplan et al (2020) give too much importance to model size.
For instance, the resources used for Gopher (280B) should have been used to train a 40-70B parameters model



6.7 Approach 1 (fixed-sized models)

The first approach is empirecal, training the model. Theu take different model with different sizes and trainig tham with different computer budget.

6.8 Chinchilla

DeepMind's presented Chinchilla, a 70B parameters model decoder-only architecture. Trained on the same compute budget as Gopher (280B parameters). Chinchilla is $\frac{1}{4}$ of Gopher's size and is trained on more tokens (1.4T Chinchilla vs 0.3T Gopher). The paper shows how Chinchilla generally outperforms Gopher, but also GPT-3, on various tasks.

The new trend of prevous trend "always larger", is now smalled and trained for longer.

6.9 Llama family

Llama (Large Language Model Meta AI) is a family of models introduced by Meta AI, starting in 2023. All autoregressive, decoder-only architectures, trained on open datasets, available in HuggingFace.

Capitolo 7

Metrics

7.1 Geometric mean (a digression)

The geometric mean over values $x_1, x_2, \ldots, x_N$ computes a mean across N values

$$GM = (x_1 \cdot x_2 \cdot \ldots \cdot x_N)^{\frac{1}{N}} = \prod_i^N x_i^{\frac{1}{N}}$$

GM penalizes more (w.r.t .arithmetic mean) the presence of low values:

- Arithmetic mean (0.1, 0.9, 0.9) = 0.633
- Geometric mean (0.1, 0.9, 0.9) = 0.433

Computing the product of many values can produce numerical instability. So we often find, instead:

$$GM = \exp(\log(\prod_i^N x_i)) = \exp(\frac{1}{N}\sum_i^N \ln(x_i))$$

7.2 Perplexity

Perplexity is a metric that quantifies how uncertain the model is in predicting the (correct) next word

- **High perplexity**: the model is uncertain about the "correct" next word
- **Low perplexity**: the model is certain about the "correct" next word

It is quantified in terms of how large the probability assigned to each next word of a sentence is.
The number represents the average [1] number of words the model is indecise over, for a sentence.

Example The dog chased the ...
Probability distribuAon:

- P(cat) = 0.25
- P(mouse) = 0.15
- P(table) = 0.05
- P(chair) = 0.1
- P(computer) = 0.05
- P(tger) = 0.2

[1] Geometric mean

- P(crocodile) = 0.1
- P(owner) = 0.1
- P(the) = 0

Correct word: cat: P(cat) = 0.25 Even if the vocabulary has 10 words, It's as if the model was confused between 4 words (1/0.25 = 0.25-1), each with equal probability